

Introduction to Data Science

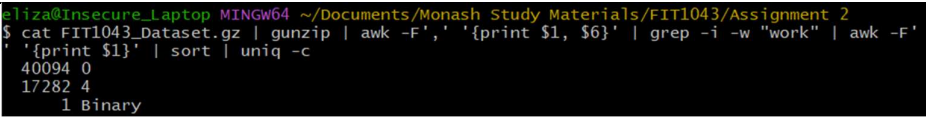
Data Analysis using Bash and R

A1		
1.	Code	<code>ls -lh FIT1043_Dataset.gz</code>
	Output	<pre>eozy@InsecureLaptop:~\$ ls -lh FIT1043_Dataset.gz -rw-r--r-- 1 eozy eozy 74M May 17 23:38 FIT1043_Dataset.gz</pre>
	Answer	The size of FIT1043_Dataset.gz file is 74 Megabytes, and the command used is 'ls -lh' to show the size of the file in a human readable form.
2.	Process Code	<code>cat FIT1043_Dataset.gz gunzip less</code> /<tab> /,
	Final Code	<code>cat FIT1043_Dataset.gz gunzip less</code> /,
	Process	The 'cat' is used to load the file into the terminal, and 'gunzip' is used to decompress the loaded content temporarily. This is to preserve the original dataset without permanently decompressing it. The 'less' command is used to see all the data to observe the field delimiter. It is observed to have ',' as the delimiter and '/', is entered for further confirmation of the delimiter used.
	Output	<pre>0_1467810672,Mon Apr 06 22:19:49 PDT 2009,NO_QUERY,scotthamilton, is upset tha e his Facebook by texting it... and might cry as a result School today also. 0_1467810917,Mon Apr 06 22:19:53 PDT 2009,NO_QUERY,mattycus,@Kenichan I dived the ball. Managed to save 50% The rest go out of bounds 0_1467811184,Mon Apr 06 22:19:57 PDT 2009,NO_QUERY,ElleCTF,my whole body feel its on fire 0_1467811193,Mon Apr 06 22:19:57 PDT 2009,NO_QUERY,karoli,@nationwideclass no ng at all. i'm mad. why am i here? because I can't see you all over there. 0_1467811372,Mon Apr 06 22:20:00 PDT 2009,NO_QUERY,joy_wolf,@Kwesidei not the 0_1467811592,Mon Apr 06 22:20:03 PDT 2009,NO_QUERY,mybirch,Need a hug 0_1467811594,Mon Apr 06 22:20:03 PDT 2009,NO_QUERY,cozz,@LOLTrish hey long t . Rains a bit only a bit LOL I'm fine thanks how's you ? 0_1467811795,Mon Apr 06 22:20:05 PDT 2009,NO_QUERY,2Hood4Hollywood,@Tatiana_K t have it 0_1467812025,Mon Apr 06 22:20:09 PDT 2009,NO_QUERY,mimismo,@twittera que me m 0_1467812416,Mon Apr 06 22:20:16 PDT 2009,NO_QUERY,erinx3leannexo,spring brea .. it's snowing 0_1467812579,Mon Apr 06 22:20:17 PDT 2009,NO_QUERY,pardonlauren,I just re-pie 0_1467812723,Mon Apr 06 22:20:19 PDT 2009,NO_QUERY,TLeC,@caregiving I couldn' it. And I thought the UA loss was embarrassing 0_1467812771,Mon Apr 06 22:20:19 PDT 2009,NO_QUERY,robrobberobert,@octolinz1 dk why I did either. you never talk to me anymore 0_1467812784,Mon Apr 06 22:20:20 PDT 2009,NO_QUERY,bayofwolves,@smarrison i w first but i didn't have a gun. not really though zac snyder's just a douc 0_1467812964,Mon Apr 06 22:20:22 PDT 2009,NO_QUERY,lovesongwriter,Hollis' dea</pre>
	Answer	Thus, it is concluded that the dataset uses ',' as its delimiter.
3.	Code	<code>cat FIT1043_Dataset.gz gunzip wc -l</code>

	Process	The 'wc -l' command is used to get the number of lines.
	Output	<pre>eliza@Insecure_Laptop MINGW64 ~/Documents/Monash Study Materials/FIT1043/Assignment 2 \$ cat FIT1043_Dataset.gz gunzip wc -l 1471793</pre>
	Answer	There are 1471793 lines in the dataset.
A2		
1.	Process Code	<pre>cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$5}' uniq -uc less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$5}' uniq -c sort less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$5}' uniq -uc sort less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$5}' sort uniq less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$5}' sort uniq -c less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$5}' sort uniq -c wc -l</pre>
	Final Code	<pre>cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$5}' sort uniq -c wc -l</pre>
	Process	The 'awk' command is used to select the specific 'user' column ('{print \$5}'), separated with the delimiter ',', to be processed with the following operations. The 'uniq' command is used to select the unique values without duplicates, with -c to count how many times the same value appeared in the dataset. I used 'sort' to sort the list of users in ascending order and group the identical usernames together. Then, to count the number of unique users, the 'wc -l' is used to count the number of lines of the list, which corresponds to the number of unique users.
	Output	<pre>eliza@Insecure_Laptop MINGW64 ~/Documents/Monash Study Materials/FIT1043/Assignment 2 \$ cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$5}' sort uniq -c wc -l 626684</pre>
	Answer	There are 626684 unique users in the dataset.
2.	Code	<pre>cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$3}' head -1 cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$3}' tail -1</pre>

	Process	This can be observed by printing out the first row and the last row of the date data, since we can assume the data is ordered chronologically. The ‘awk -F ‘,’ ‘{print \$3}’ command is used to select the date column. Then the ‘head -1’ command is to output one head row of the dataset, while ‘tail -1’ does the same but outputs one tail row (last row) of the dataset.
	Output	<pre> eliza@Insecure_Laptop MINGW64 ~/Documents/Monash Study Materials/FIT1043/Assignment 2 \$ cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$3}' head -1 Mon Apr 06 22:19:49 PDT 2009 eliza@Insecure_Laptop MINGW64 ~/Documents/Monash Study Materials/FIT1043/Assignment 2 \$ cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$3}' tail -1 Tue Jun 16 08:40:50 PDT 2009 </pre>
	Answer	The date range for the Twitter post is from 6 th April 2009 to 16 th June 2009.
3.	Code	<pre> cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$3}' awk -F' ' '{print \$2, \$3, \$6}' sort -M uniq less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$3}' awk -F' ' '{print \$2, \$3, \$6}' sort -M uniq head -1 cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$3}' awk -F' ' '{print \$2, \$3, \$6}' sort -M uniq tail -1 </pre>
	Process	I first select the original date time column from the dataset, then ‘awk’ is used again to extract the date in the format of Month Date Year for comparing. The ‘sort -M’ command is used to sort the list of extracted dates following the Monthly order. ‘uniq’ is used to make sure no duplicates. Finally, similar to the previous question, I used the ‘head -1’ and ‘tail -1’ commands to get the starting date and the last date from the list to determine the minimum and maximum timestamps.

	Output	<pre>eliza@Insecure_Laptop MINGW64 ~/Documents/Monash Study Materials/FIT1043/Assignment 2 \$ cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$6}' grep -i -w 'France' wc -l 1313</pre>
	Answer	The number of tweets containing the word 'France', ignoring the case, is 1313.
4b.	Code	<pre>cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$6}' grep -i -w "France" grep -v "France" grep -v "france" grep -v "FRANCE"</pre>
	Process	First, I extracted all the tweet content containing 'France' by 'grep -i -w', ignoring the case and ensuring its standalone, similar to Question 4a. Then, I excluded the tweets with the keywords 'France', 'france', and 'FRANCE' from the extracted tweets using 'grep -v'. Thus, this resulted in retaining tweets that contain 'france' keyword but excluded all tweets that contain 'france' in the form of 'France', 'france', and 'FRANCE'.
	Output	<pre>eliza@Insecure_Laptop MINGW64 ~/Documents/Monash Study Materials/FIT1043/Assignment 2 \$ cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$6}' grep -i -w 'France' grep -v " france" grep -v "france" grep -v "FRANCE" @Thea1992 Watch the news ... on N24 or N-TV ... a Air France flight is missed ... and nobody know where it is Binary file (standard input) matches</pre>
	Answer	There is only 1 tweet containing 'France' with combinations of uppercase and lowercase, after excluding the forms "France", "france", "FRANCE".
A3		
1.	Process Code	<pre>cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$6}' grep -i "coffee" less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$6}' grep -i -w "coffee" awk -F',' '{print \$1}' less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$6}' grep -i -w "coffee" awk -F' ' '{print \$1}' sort uniq -c</pre>
	Final Code	<pre>cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$6}' grep -i -w "coffee" awk -F' ' '{print \$1}' sort uniq -c</pre>
	Process	Firstly, the polarity and tweet text column are selected (awk -F',' '{print \$1, \$6}') for the searching. The 'grep -i -w' command is used to find the tweets that contain the standalone word 'coffee' and ignores the case, similar to Question A2.4b. Then, the sentiment column (awk -F' ' '{print \$1}') is selected for the counting operations. The 'awk' command is used to select the polarity column of the tweet for counting later. 'uniq -c' is used to count the number of times each polarity value appeared in the dataset.
	Output	<pre>eliza@Insecure_Laptop MINGW64 ~/Documents/Monash Study Materials/FIT1043/Assignment 2 \$ cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$6}' grep -i -w "coffee" awk -F ' ' '{print \$1}' sort uniq -c 2311 0 3662 4</pre>
	Answer	Tweets mentioned the word 'coffee': Negative (0) = 2311

		Positive (4) = 3662
2.	Code	<code>cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$6}' grep -i -w "work" awk -F' ' '{print \$1}' sort uniq -c</code>
	Process	The method used here is similar to that used in Question A3.1, but the keyword used for 'grep -I -w' is changed from 'coffee' to 'work' to filter the tweets.
	Output	
	Answer	Tweets mentioned the word 'work': Negative (0) = 40094 Positive (4) = 17282
3.	Process Code	<pre> coffeeN=\$(cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1,\$6}' grep -i -w "coffee" awk -F' ' '\$1==0 {print \$1}' wc -l) coffeeP=\$(cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1,\$6}' grep -i -w "coffee" awk -F' ' '\$1==4 {print \$1}' wc -l) echo "\$coffeeN" echo "\$coffeeP" echo "Keyword,Negative,Positive" > sentiment_comparison.csv echo "coffee,\$coffeeN,\$coffeeP" >> sentiment_comparison.csv head -2 sentiment_comparison.csv less workN=\$(cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1,\$6}' grep -i -w "work" awk -F' ' '\$1==0 {print \$1}' wc -l) workP=\$(cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1,\$6}' grep -i -w "work" awk -F' ' '\$1==4 {print \$1}' wc -l) echo "\$workN" echo "\$workP" echo "work,\$workN,\$workP" >> sentiment_comparison.csv head -3 sentiment_comparison.csv less </pre>

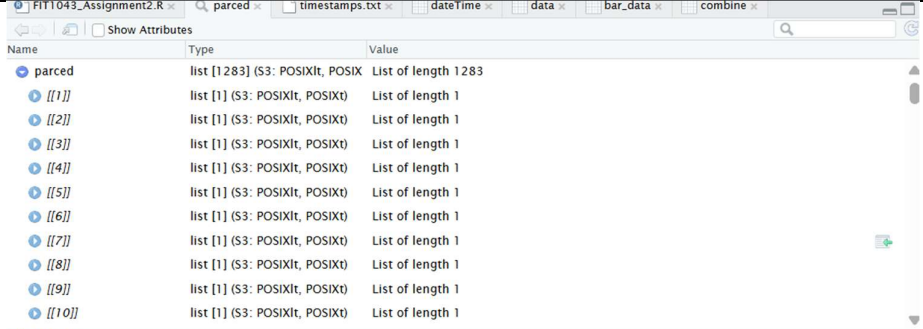
	Final Code	<pre> coffeeN=\$(cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1,\$6}' grep -i -w "coffee" awk -F' ' '\$1==0 {print \$1}' wc -l) coffeeP=\$(cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1,\$6}' grep -i -w "coffee" awk -F' ' '\$1==4 {print \$1}' wc -l) echo "Keyword,Negative,Positive" > sentiment_comparison.csv echo "coffee,\$coffeeN,\$coffeeP" >> sentiment_comparison.csv workN=\$(cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1,\$6}' grep -i -w "work" awk -F' ' '\$1==0 {print \$1}' wc -l) workP=\$(cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1,\$6}' grep -i -w "work" awk -F' ' '\$1==4 {print \$1}' wc -l) echo "work,\$workN,\$workP" >> sentiment_comparison.csv </pre>
	Process	The implementation to get the tweet counts containing keywords ‘coffee’ and ‘work’ respectively is similar to Question A3.2. To make writing into a file convenient, I used variables to store the number of tweets of ‘coffee’ and ‘work’ by using ‘\$()’ command. Then I print out the variables to ensure they are correct. After that, I used ‘>’ to create or overwrite the CSV header, then the data was written to the sentiment_comparison.csv file using ‘>>’, which appends a new line instead of overwriting the whole file. Reference: The implementation of ‘\$()’ is referred to the Reference 1 and writing files operations are referred to Reference 2.
	Output	<pre> Keyword,Negative,Positive coffee,2311,3662 (END) Keyword,Negative,Positive coffee,2311,3662 work,40094,17282 (END) </pre>
	Answer	The sentiment_comparison.csv file contains: <pre> Keyword,Negative,Positive coffee,2311,3662 work,40094,17282 </pre>

4.	R Code	<pre> data <- read.table("sentiment_comparison.csv", header=TRUE,sep=',') str(data) negative_data <- data\$Negative positive_data <- data\$Positive bar_data <- matrix(rbind(negative_data,positive_data), nrow = 2, ncol = 2, byrow = TRUE) label_x <- c("Negative","Positive") keyword <- c(data\$Keyword) colors = c("lightblue","orange") plot <- barplot(bar_data, main = "Tweet Counts by Sentiment and Keyword", names.arg = label_x, xlab = "Sentiment", ylab = "Tweet Count", col = colors, beside = TRUE, ylim = c(0,50000)) legend("topright", keyword, cex = 0.8, fill = colors) text(x = plot, y = bar_data + 1, labels = bar_data, pos = 3, cex = 1.0, col = "black") </pre>
	Process	A bar chart is plotted using ‘barplot’ in R, where each sentiment category (Negative, Positive) has 2 bars for the keywords ‘coffee’ and ‘work’, respectively. The data was read from the sentiment_comparison.csv file using ‘read.table()’ with ‘header = TRUE’ and ‘sep ‘,’’ so that the CSV column can be recognised correctly. Then, the negative and positive tweet counts are combined using ‘rbind’ to group the vectors into rows. While the ‘matrix’ is used to restructure the vectors into a 2x2 matrix format suitable for plotting a grouped bar chart according to the values. The ‘names.arg’ parameter labels the x-axis categories as “Negative” and “Positive”, while ‘col’ assigns colours to the bars. The argument beside=TRUE is important because it places the bars side-by-side for easier comparison instead of stacking them together. In addition, ‘legend()’ is used to identify which colour corresponds to each keyword, while text() is used to display the exact tweet counts above each bar to improve the readability of the visualisation. Reference: The grouped bar chart code implementation is referred to Reference 3. The ‘matrix’ and ‘rbind’ methods are referred to Reference 4 for better understanding.

	Output	Tweet Counts by Sentiment and Keyword <table border="1"> <thead> <tr> <th>Sentiment</th> <th>coffee</th> <th>work</th> </tr> </thead> <tbody> <tr> <td>Negative</td> <td>2311</td> <td>40094</td> </tr> <tr> <td>Positive</td> <td>3662</td> <td>17282</td> </tr> </tbody> </table>	Sentiment	coffee	work	Negative	2311	40094	Positive	3662	17282
Sentiment	coffee	work									
Negative	2311	40094									
Positive	3662	17282									
5.	Answer	The keyword 'coffee' appears more often (3662) in positive tweets than negative tweets (2311). While work appears a lot more often in negative tweets (40094) and lower positive tweets (17282). This is likely because coffee is usually discussed as something enjoyable or desired, while work is mainly associated with negative feelings such as tiredness, obligations, or complaints. However, this analysis does not consider the context behind the tweets containing the keyword; therefore, the sentiment patterns should be interpreted with more supporting detailed analysis.									
6.	Process Code	<pre>cat FIT1043_Dataset.gz gunzip awk -F',' '\$1==0 {print\$6}' tr -c '[:alnum:]' '\n' grep -v -w -f stopwords.txt sort uniq -c sort -nr head -10 cat FIT1043_Dataset.gz gunzip awk -F',' '\$1==4 {print\$6}' tr -c '[:alnum:]' '\n' grep -v -w -f stopwords.txt sort uniq -c sort -nr head -10 cat FIT1043_Dataset.gz gunzip awk -F',' '\$1==0 {print\$6}' tr 'A-Z' 'a-z' tr -cs '[:alnum:]' '\n' grep -v -w -f stopwords.txt sort uniq -c sort -nr head - 10 cat FIT1043_Dataset.gz gunzip awk -F',' '\$1==4 {print\$6}' tr 'A-Z' 'a-z' tr -cs '[:alnum:]' '\n' grep -v -w -f stopwords.txt sort uniq -c sort -nr head - 10 cat FIT1043_Dataset.gz gunzip awk -F',' '\$1==0 {print\$6}' tr 'A-Z' 'a-z' tr -cs '[:alpha:]' '\n' grep</pre>									

		<pre>-v -w -f stopwords.txt grep -v 'http' grep -v 'com' awk 'length>1' sort uniq -c sort -nr head -10</pre> <pre>cat FIT1043_Dataset.gz gunzip awk -F',' '\$1==4 {print\$6}' tr 'A-Z' 'a-z' tr -cs '[:alpha:]' '\n' grep -v -w -f stopwords.txt grep -v 'http' grep -v 'com' awk 'length>1' sort uniq -c sort -nr head -10</pre>
	Final Code	<pre>cat FIT1043_Dataset.gz gunzip awk -F',' '\$1==0 {print\$6}' tr 'A-Z' 'a-z' tr -cs '[:alpha:]' '\n' grep -v -w -f stopwords.txt grep -v 'http' grep -v 'com' awk 'length>1' sort uniq -c sort -nr head -10</pre> <pre>cat FIT1043_Dataset.gz gunzip awk -F',' '\$1==4 {print\$6}' tr 'A-Z' 'a-z' tr -cs '[:alpha:]' '\n' grep -v -w -f stopwords.txt grep -v 'http' grep -v 'com' awk 'length>1' sort uniq -c sort -nr head -10</pre>
	Process	For the searching process, I used ‘awk -F',' '\$1==4 {print\$6}’ to extract either the positive (4) or negative (0) tweets separately. It filters the dataset based on the sentiment in column \$1 with 0 or 4 and outputs the tweet text from column \$6. Next, the ‘tr 'A-Z' 'a-z’ is used to convert every word into lowercase so that the words with different capitalisation such as ‘coffee’ and ‘Coffee’, would be grouped as the same word correctly. Then, ‘tr -cs '[:alpha:]' '\n’ is applied to split the tweet sentences into individual words. The ‘alpha’ means alphabetic characters only. Then, the ‘tr -c’ will replace any elements that are not alphabetic characters with ‘\n’, while ‘-s’ squeezes repeated replacement into a single line. This transforms the tweet content into a line-by-line list of words. Initially, ‘alnum’ was used, which included both alphabetic characters and numbers, but later I replaced it with ‘alpha’ because the task focuses on identifying words rather than numbers. Besides, I used ‘grep -v -w -f stopwords.txt’ to remove (-v) common stop words that matched exactly (-w) the words in the stopwords.txt file. The ‘-f’ allows grep to read the stopwords directly from the stopwords.txt file. Moreover, I used ‘grep -v 'http' grep -v 'com’ to remove URL format lines generated after splitting, as they are not meaningful words. I also applied the ‘awk 'length>1’ to exclude lines that contain only single characters like ‘m’ and ‘u’, as they are not valid words as well. Lastly, the ‘sort uniq -c’ command is used to sort the result and count the frequency of the words. Then ‘sort -nr head -10’ is used to sort the counted result based on their frequency numerically to get the correct descending

		ordering for the most frequent words, after that ‘head -10’ will display the Top 10 most frequent words. Reference: The implementation of searching is referred to examples from References 5 and 6. Reference 7 is referred to for the use of removing single-character lines.	
	Output	<pre> eliza@Insecure_Laptop MINGW64 ~/Documents/Monash Study Materials/FIT1043/Assignm ent 2 \$ cat FIT1043_Dataset.gz gunzip awk -F',' '\$1==0 {print\$6}' tr 'A-Z' 'a-z' tr -cs '[: alpha:]' '\n' grep -v -w -f stopwords.txt grep -v -w 'm' grep -v 'http' grep -v 'com' awk 'length>1' sort uniq -c sort -nr head -10 41961 go 41830 get 41773 work 38067 day 37801 like 34968 today 30972 going 30690 got 30377 back 29006 really </pre> <pre> eliza@Insecure_Laptop MINGW64 ~/Documents/Monash Study Materials/FIT1043/Assignment 2 \$ cat FIT1043_Dataset.gz gunzip awk -F',' '\$1==4 {print\$6}' tr 'A-Z' 'a-z' tr -cs '[: alpha:]' '\n' grep -v -w -f stopwords.txt grep -v -w 'm' grep -v 'http' grep -v 'com' awk 'length>1' sort uniq -c sort -nr head -10 56977 good 44446 day 43846 love 42203 quot 34546 like 33685 get 33182 lol 31774 thanks 28414 going 28063 time </pre>	
	Answer	Negative Tweets Top 10 Most Frequent Words: 1. go 2. get 3. work 4. day 5. like 6. today 7. going 8. got 9. back 10. really	Positive Tweets Top 10 Most Frequent Words: 1. good 2. day 3. love 4. quot 5. like 6. get 7. lol 8. thanks 9. going 10. time
A4			
1.	Process Code	<pre> cat FIT1043_Dataset.gz gunzip awk -F',' '{print\$3, \$6}' grep -i -w 'australia' sort wc -l cat FIT1043_Dataset.gz gunzip awk -F',' '{print\$3, \$6}' grep -i -w 'australia' awk -F' ' '{print \$1, \$2, \$3, \$6, \$4}' wc -l cat FIT1043_Dataset.gz gunzip awk -F',' '{print\$3, \$6}' grep -i -w 'australia' awk -F' ' '{print \$1, \$2, \$3, \$6, \$4}' less </pre>	

		<pre>\$ cat FIT1043_Dataset.gz gunzip awk -F',' '{print\$3,\$6}' grep -i -w 'australia' awk -F' ' '{print \$1, \$2,\$3, \$6, \$4}' sort -M > timestamps.txt</pre>																																				
	Final Code	<pre>\$ cat FIT1043_Dataset.gz gunzip awk -F',' '{print\$3,\$6}' grep -i -w 'australia' awk -F' ' '{print \$1, \$2,\$3, \$6, \$4}' sort -M > timestamps.txt</pre>																																				
	Output	<pre>sliza@Insecure_Laptop MINGW64 ~/Documents/Monash Study Materials/FIT1043/Assignment 2 \$ cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$3, \$6}' grep -i -w "australia" wc -l 1284</pre> <pre>Fri Apr 17 2009 21:32:22 Fri Apr 17 2009 22:17:43 Fri Jun 05 2009 09:51:27 Fri Jun 05 2009 10:20:27 Fri Jun 05 2009 11:04:43 Fri Jun 05 2009 11:43:17 Fri Jun 05 2009 12:04:17 Fri Jun 05 2009 13:40:59 Fri Jun 05 2009 14:49:17 Fri Jun 05 2009 14:51:28 Fri Jun 05 2009 15:33:01 Fri Jun 05 2009 16:39:10 Fri Jun 05 2009 16:55:29 Fri Jun 05 2009 16:59:24 Fri Jun 05 2009 17:01:47 Fri Jun 05 2009 17:02:04 Fri Jun 05 2009 17:15:23 Fri Jun 05 2009 17:57:46</pre>																																				
	Answer	I chose 'Australia' for this task. I used 'grep -i -w' to search for 'australia' without case sensitivity from the tweet text column (awk -F',' '{print\$3, \$6}'), and it must be a standalone word using '-w' as 'whole'. Then I split them using 'awk -F ' ' ' to select only the weekday, date, year, and time to store into the timestamps.txt file. Additionally, I used 'sort -M' to organise the extracted timestamps before writing them into the file.																																				
2.	Code	<pre>dateTime <-readLines("timestamps.txt") parsed <- strptime(dateTime, "%a %b %d %Y %H:%M:%S")</pre>																																				
	Process	The date and time are read from the timestamps.txt file. Then, they are converted from plain text into formatted objects based on their order. The extracted timestamp format in the timestamps.txt file is weekday name, Month name, Integer Date, Integer Year, and time in the format of 00:00:00. The 'strptime()' function is used for the conversion. Reference: This implementation is referred to the source provided from the guideline (Reference 8).																																				
	Output	 The screenshot shows the RStudio environment with several tabs open: 'FIT1043_Assignment2.R', 'parsed', 'timestamps.txt', 'dateTime', 'data', 'bar_data', and 'combine'. The 'parsed' tab is active, displaying a list of 1283 POSIXt objects. The table below represents the data shown in the screenshot: <table border="1"> <thead> <tr> <th>Name</th> <th>Type</th> <th>Value</th> </tr> </thead> <tbody> <tr> <td>parsed</td> <td>list [1283] (S3: POSIXt, POSIXt)</td> <td>List of length 1283</td> </tr> <tr> <td>[[1]]</td> <td>list [1] (S3: POSIXt, POSIXt)</td> <td>List of length 1</td> </tr> <tr> <td>[[2]]</td> <td>list [1] (S3: POSIXt, POSIXt)</td> <td>List of length 1</td> </tr> <tr> <td>[[3]]</td> <td>list [1] (S3: POSIXt, POSIXt)</td> <td>List of length 1</td> </tr> <tr> <td>[[4]]</td> <td>list [1] (S3: POSIXt, POSIXt)</td> <td>List of length 1</td> </tr> <tr> <td>[[5]]</td> <td>list [1] (S3: POSIXt, POSIXt)</td> <td>List of length 1</td> </tr> <tr> <td>[[6]]</td> <td>list [1] (S3: POSIXt, POSIXt)</td> <td>List of length 1</td> </tr> <tr> <td>[[7]]</td> <td>list [1] (S3: POSIXt, POSIXt)</td> <td>List of length 1</td> </tr> <tr> <td>[[8]]</td> <td>list [1] (S3: POSIXt, POSIXt)</td> <td>List of length 1</td> </tr> <tr> <td>[[9]]</td> <td>list [1] (S3: POSIXt, POSIXt)</td> <td>List of length 1</td> </tr> <tr> <td>[[10]]</td> <td>list [1] (S3: POSIXt, POSIXt)</td> <td>List of length 1</td> </tr> </tbody> </table> (No selection)	Name	Type	Value	parsed	list [1283] (S3: POSIXt, POSIXt)	List of length 1283	[[1]]	list [1] (S3: POSIXt, POSIXt)	List of length 1	[[2]]	list [1] (S3: POSIXt, POSIXt)	List of length 1	[[3]]	list [1] (S3: POSIXt, POSIXt)	List of length 1	[[4]]	list [1] (S3: POSIXt, POSIXt)	List of length 1	[[5]]	list [1] (S3: POSIXt, POSIXt)	List of length 1	[[6]]	list [1] (S3: POSIXt, POSIXt)	List of length 1	[[7]]	list [1] (S3: POSIXt, POSIXt)	List of length 1	[[8]]	list [1] (S3: POSIXt, POSIXt)	List of length 1	[[9]]	list [1] (S3: POSIXt, POSIXt)	List of length 1	[[10]]	list [1] (S3: POSIXt, POSIXt)	List of length 1
Name	Type	Value																																				
parsed	list [1283] (S3: POSIXt, POSIXt)	List of length 1283																																				
[[1]]	list [1] (S3: POSIXt, POSIXt)	List of length 1																																				
[[2]]	list [1] (S3: POSIXt, POSIXt)	List of length 1																																				
[[3]]	list [1] (S3: POSIXt, POSIXt)	List of length 1																																				
[[4]]	list [1] (S3: POSIXt, POSIXt)	List of length 1																																				
[[5]]	list [1] (S3: POSIXt, POSIXt)	List of length 1																																				
[[6]]	list [1] (S3: POSIXt, POSIXt)	List of length 1																																				
[[7]]	list [1] (S3: POSIXt, POSIXt)	List of length 1																																				
[[8]]	list [1] (S3: POSIXt, POSIXt)	List of length 1																																				
[[9]]	list [1] (S3: POSIXt, POSIXt)	List of length 1																																				
[[10]]	list [1] (S3: POSIXt, POSIXt)	List of length 1																																				

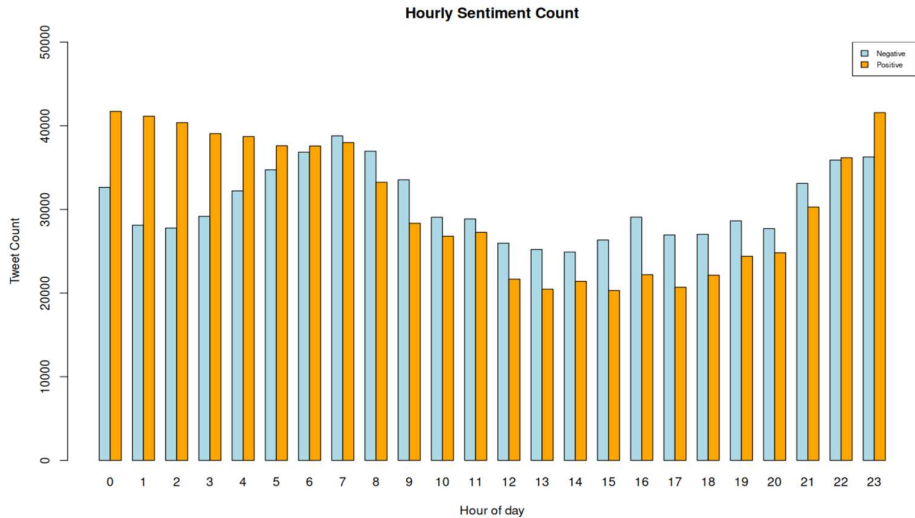
	Answer	This can be matched with the format: %a %b %d %Y %H:%M:%S. Where %a is for the abbreviated weekday name, %b is for the abbreviated month name, and %Y corresponds to 4-digit year. Then %H is for hours (00-23), %M is minutes as decimal numbers and lastly %S for seconds as integers.																						
3.	R Code	<pre>dates <- as.Date(parsed) tweetPerDay <- table(dates) hist(tweetPerDay, main = "Distribution of Daily Tweet Counts Containing 'Australia'", xlab = "Number of Tweets Per Day", ylab = 'Number of Days', col = "steelblue", border = "white" , labels = TRUE)</pre>																						
	Process	‘as.Date()’ is used to extract the date component from the parsed date-time object, which allows the tweets to be grouped based on days rather than exact timestamps. Next, ‘table()’ is used to get the frequency of dates that each occurrence represents each tweet, resulting in daily tweet number values. Finally, I used the hist() function to visualise the distribution of daily tweet counts. Reference: The use ‘as.Date’ is referred to Reference 9, while ‘table()’ can be referred to Reference 10.																						
	Output	Distribution of Daily Tweet Counts Containing 'Australia' <table><thead><tr><th>Number of Tweets Per Day</th><th>Number of Days</th></tr></thead><tbody><tr><td>0-10</td><td>14</td></tr><tr><td>10-20</td><td>10</td></tr><tr><td>20-30</td><td>6</td></tr><tr><td>30-40</td><td>4</td></tr><tr><td>40-50</td><td>4</td></tr><tr><td>50-60</td><td>2</td></tr><tr><td>60-70</td><td>2</td></tr><tr><td>70-80</td><td>1</td></tr><tr><td>80-90</td><td>1</td></tr><tr><td>90-100</td><td>2</td></tr></tbody></table>	Number of Tweets Per Day	Number of Days	0-10	14	10-20	10	20-30	6	30-40	4	40-50	4	50-60	2	60-70	2	70-80	1	80-90	1	90-100	2
Number of Tweets Per Day	Number of Days																							
0-10	14																							
10-20	10																							
20-30	6																							
30-40	4																							
40-50	4																							
50-60	2																							
60-70	2																							
70-80	1																							
80-90	1																							
90-100	2																							

	Answer	The histogram visualises the distribution of daily tweet counts that contain the word 'Australia' (ignore case). The x-axis represents the number of tweets per day, while the y-axis represents the number of days that fall within the tweet-count range, such as 0-10 and 30-40. The total number of days recorded is 46 days. This shows the distribution of the number of tweets across different days. From the histogram, we can observe that more than half of the days (24 days) contain fewer than 20 tweets mentioning Australia, while only a small number of days exceed 60 tweets. The histogram appears to be a right-skewed distribution, indicating that large number of tweet activity per day is uncommon.
A5		
1.	Process Code	<pre>cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$3}' less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$3}' awk -F' ' '{print \$1, \$5}' less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$3}' awk -F' ' '{print \$1, \$5}' sed 's/[:]/ /g' less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$3}' awk -F' ' '{print \$1, \$5}' sed 's/[:]/ /g' awk -F' ' '{print \$1, \$2}' less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$3}' awk -F' ' '{print \$1, \$5}' sed 's/[:]/ /g' awk -F' ' '{print \$2, \$1}' less cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$3}' awk -F' ' '{print \$1, \$5}' sed 's/[:]/ /g' awk -F' ' '{print \$2, \$1}' sort uniq -c cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$3}' awk -F' ' '{print \$1, \$5}' sed 's/[:]/ /g' awk -F' ' '{print \$2, \$1}' sort uniq -c > temporary.txt echo "Hour,Negative,Positive" > hourly_sentiment.csv awk -F' ' 'BEGIN { if(\$3==0) {negative[\$2] = \$1} if(\$3==4){positive[\$2] = \$1}} END{for(i=0, i < 24, i++){print "\$i,\$negative[i],\$positive[i]}}' temporary.txt awk '{ if(\$3==0) {negative[\$2] = \$1} if(\$3==4){positive[\$2] = \$1}} END{for(i=0; i < 24; i++) print i,"negative[i]","positive[i]}' temporary.txt</pre>

		<pre>awk '{ hour = \$2+0; if(\$3==0) {negative[hour] = \$1} if(\$3==4){positive[hour] = \$1}} END{for(i=0; i < 24; i++) printf "%02d,%d,%d\n", i,negative[i]+0,positive[i]+0}' temporary.txt awk '{ hour = \$2+0; if(\$3==0) {negative[hour] = \$1} if(\$3==4){positive[hour] = \$1}} END{for(i=0; i < 24; i++) printf "%02d,%d,%d\n", i,negative[i]+0,positive[i]+0}' temporary.txt >> hourly_sentiment.csv head -30 hourly_sentiment.csv less</pre>
	Final Code	<pre>cat FIT1043_Dataset.gz gunzip awk -F',' '{print \$1, \$3}' awk -F' ' '{print \$1, \$5}' sed 's/[:]/ /g' awk -F' ' '{print \$2, \$1}' sort uniq -c > temporary.txt echo "Hour,Negative,Positive" > hourly_sentiment.csv awk '{ hour = \$2+0; if(\$3==0) {negative[hour] = \$1} if(\$3==4){positive[hour] = \$1}} END{for(i=0; i < 24; i++) printf "%02d,%d,%d\n", i,negative[i]+0,positive[i]+0}' temporary.txt >> hourly_sentiment.csv</pre>
	Process	Firstly, the sentiment and timestamps columns are extracted using ‘awk’ and ‘print{\$1, \$3}’, then another ‘awk’ operation (‘print \$1, \$5’) was used to extract only the sentiment value and the time component from the previous output. Throughout the process, ‘less’ is frequently used to observe the immediate outputs and ensure they match the expected output. Next, ‘sed 's/[:]/ /g' is used to replace the colon ‘:’ contained in the time format with spaces for ‘awk’ selection use, which is extracting the sentiment and the hour values (‘print \$1, \$2’). Then it is reordered so that the hour becomes the first column, making the ‘sort uniq -c’ operation easier to perform. Now, the number of negative and positive tweets for each hour is obtained and ready for restructuring. To simplify the working, I stored the counted result in a file named temporary.txt. Before restructuring the result, the header (Hour,Negative,Positive) is first written into the hourly_sentiment.csv file. I attempted to use more advanced ‘awk’ operation involving arrays, loops and conditions to restructure the counted results into the required format. Initially, I just learned the new ‘awk’ operation, so it didn’t work as I thought; several syntax and logic errors occurred. For instance, ‘BEGIN’

		was used incorrectly, as it is for operations before reading the lines in the file; I removed it because I was trying to do something to the data in the file line by line, which should happen after the BEGIN operation. I also mistakenly used commas (,) instead of semicolons (;) within the for loop syntax. Eventually, there are 2 arrays created: the ‘negative’ array and ‘positive’ array. They store the tweet counts corresponding to each hour and the hour values were used as their indices. Then, a for loop is used to print them in the format of "index in 2 numbers, negative tweets, positive tweets\n" as required in the guideline. Moreover, I added ‘hour = \$2 + 0’ to convert the hour values into integers. This simplifies the indexing and following looping operations. The stored values in the arrays are also converted into integers during output to make the result cleaner and easier for further analysis. Finally, after ensuring the result is correct, they are appended into the hourly_sentiment.csv file. Reference: The implementation of ‘sed’ is referred to the examples from Reference 11. The ‘awk’ operations are learned from Reference 12.
	Output	<pre>Hour,Negative,Positive 00,32643,41721 01,28115,41141 02,27774,40373 03,29177,39061 04,32211,38708 05,34730,37612 06,36842,37579 07,38798,37991 08,36959,33248 09,33540,28348 10,29064,26801 11,28860,27261 12,25965,21662 13,25216,20463 14,24903,21413 15,26350,20306 16,29078,22201 17,26947,20699 18,27028,22141 19,28632,24395 20,27706,24821 21,33106,30283 22,35900,36178 23,36274,41569</pre>
	Answer	The counted result is restructured successfully and is saved into the hourly_sentiment.csv file.
2.	Code	<pre>cat hourly_sentiment.csv less cat hourly_sentiment.csv tail -24 sort uniq wc -l</pre>

	Process	Firstly, the results written in the file are output to be observed. I excluded the header and sorted it using ‘sort uniq’ to ensure no duplicates. Then, ‘wc -l’ is used to output the line count to check if there are 24 hours in 24 lines of tweet counts.
	Output	 <pre>Hour,Negative,Positive 00,32643,41721 01,28115,41141 02,27774,40373 03,29177,39061 04,32211,38708 05,34730,37612 06,36842,37579 07,38798,37991 08,36959,33248 09,33540,28348 10,29064,26801 11,28860,27261 12,25965,21662 13,25216,20463 14,24903,21413 15,26350,20306 16,29078,22201 17,26947,20699 18,27028,22141 19,28632,24395 20,27706,24821 21,33106,30283 22,35900,36178 23,36274,41569 ~ ~ ~ (END) eliza@Insecure_Laptop MINGW64 ~/Documents/Monash Study Materials/FIT1043/Assignment 2 \$ cat hourly_sentiment.csv tail -24 sort uniq wc -l 24</pre>
	R code	<pre>hourlySentiment <- read.table("hourly_sentiment.csv", header = TRUE, sep = ',') hour <- hourlySentiment\$Hour negatives <- hourlySentiment\$Negative positives <- hourlySentiment\$Positive colors <- c("lightblue","orange") legendLabel <- c("Negative", "Positive") hourlyBar <- matrix(cbind(negatives,positives), nrow = 2, ncol = 24, byrow = TRUE) plot <- barplot(hourlyBar, main = "Hourly Sentiment Count", names.arg = hour, xlab = "Hour of day", ylab = "Tweet Count", col = colors, beside = TRUE, ylim = c(0,50000)) legend("topright", legendLabel, cex = 0.6, fill = colors)</pre>
	Process	A grouped bar chart is plotted using ‘barplot’ in R for the hourly sentiment data read using ‘read.table()’ from the CSV file, similar to the implementation of Question A3.4. Then the values are extracted and stored in variables: hour, negatives and positives. Next, ‘cbind()’ is used to combine the negative and positive tweet counts by columns into a matrix (‘matrix’) structure suitable for grouped plotting in barplot(). This allows each hour to contain two bars side-by-side, representing negative and positive tweet counts respectively.

		The ‘names.arg’ parameter is used to label the x-axis with the hourly values from 00 to 23, while ‘col’ functions to assign colors to the bars. The argument beside=TRUE is used to place the bars side-by-side for easier comparison. Additionally, ylim = c(0,50000) is used to set an appropriate y-axis range so that all bars can be displayed clearly within the graph. Finally, legend() is used to display the labels for the sentiment categories (“Negative”, “Positive”) at the top-right of the chart. Reference: The use of ‘cbind’ is referred to Reference 4.																																																																											
	Output	 The chart displays the hourly distribution of tweet counts for negative and positive sentiments. The y-axis represents the 'Tweet Count' ranging from 0 to 50,000. The x-axis represents the 'Hour of day' from 0 to 23. Blue bars represent negative sentiment, and orange bars represent positive sentiment. The data shows a peak in activity around hour 7, with positive sentiment generally higher than negative sentiment during the night and early morning hours. <table border="1"> <thead> <tr> <th>Hour of day</th> <th>Negative</th> <th>Positive</th> </tr> </thead> <tbody> <tr><td>0</td><td>33000</td><td>42000</td></tr> <tr><td>1</td><td>28000</td><td>41000</td></tr> <tr><td>2</td><td>28000</td><td>40000</td></tr> <tr><td>3</td><td>29000</td><td>39000</td></tr> <tr><td>4</td><td>32000</td><td>38000</td></tr> <tr><td>5</td><td>35000</td><td>37000</td></tr> <tr><td>6</td><td>37000</td><td>36000</td></tr> <tr><td>7</td><td>38000</td><td>35000</td></tr> <tr><td>8</td><td>36000</td><td>33000</td></tr> <tr><td>9</td><td>33000</td><td>29000</td></tr> <tr><td>10</td><td>29000</td><td>27000</td></tr> <tr><td>11</td><td>28000</td><td>26000</td></tr> <tr><td>12</td><td>26000</td><td>22000</td></tr> <tr><td>13</td><td>25000</td><td>21000</td></tr> <tr><td>14</td><td>25000</td><td>22000</td></tr> <tr><td>15</td><td>26000</td><td>21000</td></tr> <tr><td>16</td><td>29000</td><td>23000</td></tr> <tr><td>17</td><td>27000</td><td>21000</td></tr> <tr><td>18</td><td>27000</td><td>22000</td></tr> <tr><td>19</td><td>29000</td><td>25000</td></tr> <tr><td>20</td><td>28000</td><td>25000</td></tr> <tr><td>21</td><td>33000</td><td>30000</td></tr> <tr><td>22</td><td>36000</td><td>35000</td></tr> <tr><td>23</td><td>36000</td><td>42000</td></tr> </tbody> </table>	Hour of day	Negative	Positive	0	33000	42000	1	28000	41000	2	28000	40000	3	29000	39000	4	32000	38000	5	35000	37000	6	37000	36000	7	38000	35000	8	36000	33000	9	33000	29000	10	29000	27000	11	28000	26000	12	26000	22000	13	25000	21000	14	25000	22000	15	26000	21000	16	29000	23000	17	27000	21000	18	27000	22000	19	29000	25000	20	28000	25000	21	33000	30000	22	36000	35000	23	36000	42000
Hour of day	Negative	Positive																																																																											
0	33000	42000																																																																											
1	28000	41000																																																																											
2	28000	40000																																																																											
3	29000	39000																																																																											
4	32000	38000																																																																											
5	35000	37000																																																																											
6	37000	36000																																																																											
7	38000	35000																																																																											
8	36000	33000																																																																											
9	33000	29000																																																																											
10	29000	27000																																																																											
11	28000	26000																																																																											
12	26000	22000																																																																											
13	25000	21000																																																																											
14	25000	22000																																																																											
15	26000	21000																																																																											
16	29000	23000																																																																											
17	27000	21000																																																																											
18	27000	22000																																																																											
19	29000	25000																																																																											
20	28000	25000																																																																											
21	33000	30000																																																																											
22	36000	35000																																																																											
23	36000	42000																																																																											
	Answer	Through the BASH command checking process, it can be confirmed that all 24 hours of tweet count data exist without duplicates or missing values. From my observation of the chart, the distribution of tweet counts is not uniform. The activity of tweets appeared to be higher and more positive in number during late-night (22-23) and early-morning hours (00-07). Positive tweets also appeared to be higher than negative tweets during some of these periods. One possible reason for this is that users generally are more active in posting tweets during personal or leisure time, especially at night when they are not engaged in work or daily responsibilities.																																																																											
3a.	Answer	One limitation of this recommendation is that the data of hourly sentiment tweet counts only contains the sentiment labels and timestamps, which do not contain any commercial engagement information that can demonstrate the users’ engagement and characteristics, such as likes, number of shares and click-rates. Therefore, it is insufficient to determine if the active tweet hours have the targeted audience or better engagement. Besides that, analysing solely on the number of labelled tweets across hours is not comprehensive because the sentiment does not consider the content, context, topic or intention of the tweets. Positive sentiment tweets may not																																																																											

		actually reflect the true users' mood, which does not guarantee marketing effectiveness. Thus, having a higher proportion of positive tweets does not necessarily mean it will be better for audience engagement.
3b.	Answer	This is a problem because different users posted tweets at different time zones across regions or countries, which may be recorded as some local time. For instance, a user posting a tweet in Malaysia at 21:00 local time and another user posting a tweet in the US at 21:00 local time are actually happening at different moments but are grouped together as 21-hour tweets. As a result, the hourly sentiment analysis does not represent an accurate and consistent timeline, which indicates that the analysis cannot reflect the real-time tweet count globally to be used for decision-making. This issue cannot be fully corrected using only the columns in the dataset. This is because there is no reliable information about the posted tweet's actual locations or time zone. Therefore, it is impossible to label or standardise all timestamps accurately into a global time zone using the existing data alone.
3c.	Answer	One decision I made in my BASH pipeline was to store the aggregated result of counted tweets in a temporary file (temporary.txt) before restructuring the result's format. I chose this approach because it made the working cleaner and debugging easier. I could focus on the reformatting commands separately from the data reading process earlier. The alternative way is that the entire process could have been done with a single long pipeline command without creating a temporary file to store the result halfway. This could reduce the use of file creation, but it can make the workflow more complex. Therefore, the trade-off of this approach is that it may reduce readability and make debugging more difficult.

Reference

1. <https://stackoverflow.com/questions/2314750/how-to-assign-the-output-of-a-bash-command-to-a-variable>
2. <https://www.redhat.com/en/blog/scripting-writing-text-file>
3. <https://www.geeksforgeeks.org/r-language/r-bar-charts/>
4. <https://www.geeksforgeeks.org/r-language/combining-matrices-in-r/>
5. <https://www.baeldung.com/linux/n-most-frequent-words-in-file>
6. <https://unix.stackexchange.com/questions/41479/find-n-most-frequent-words-in-a-file>
7. <https://stackoverflow.com/questions/21063953/delete-single-character-rows-in-a-text-file>
8. <https://www.rdocumentation.org/packages/base/versions/3.6.2/topics/strptime>
9. <https://stackoverflow.com/questions/63677684/extract-date-and-time-from-datetime-field-in-r>
10. <https://www.geeksforgeeks.org/r-language/frequency-table-in-r/>
11. <https://www.baeldung.com/linux/string-split-many-delimiters>
12. https://www.tutorialspoint.com/awk/awk_quick_guide.htm